

21. Offset-Based Pagination

Phase: L **Code:** `search_api/services/search.py` , `search_api/models/requests.py` , `admin-ui/src/pages/SearchTest.jsx`

What it is

Pagination is the technique of splitting a large result set into smaller "pages" so the user sees a manageable number of results at a time (e.g., 10 per page) with controls to move forward and backward.

Offset-based pagination is the simplest approach: the client sends two parameters with each request:

- `offset` -- how many results to skip (page 1 = offset 0, page 2 = offset 10, page 3 = offset 20, etc.)
- `limit` -- how many results to return per page

The server runs the full query, skips the first `offset` results, and returns the next `limit` results. It also returns a `total_count` so the client can calculate how many pages exist.

In SQL, this translates directly to the `OFFSET` and `LIMIT` clauses:

```
SELECT * FROM results ORDER BY score DESC OFFSET 20 LIMIT 10;  
-- Skip the first 20 results, return the next 10
```

In our system, the pagination happens at the RRF fusion level rather than in SQL -- we paginate the fused result list in Python -- but the concept is the same.

Real-life analogy

Imagine you are a librarian and a visitor asks "Show me all books about Python." You have 200 matching books.

Without pagination: You wheel out a cart with all 200 books. The visitor is overwhelmed and the cart is heavy.

With offset-based pagination: You say "I'll bring you 10 at a time." On the first trip you bring books 1-10. The visitor says "next page" and you bring books

11-20. If they say "go to page 15", you count to book 141 (offset = 140) and bring books 141-150.

The drawback: if the visitor asks for page 15, you still have to count through the first 140 books to find where page 15 starts. For a small library this is fine, but for a library with millions of books, this counting step gets slow. (This is the performance issue with offset pagination at scale -- more on that in the "known issues" section.)

Why we used it here

R15, R16, R17 all need to handle potentially large result sets. The Admin UI's search test page shows 10 results at a time with Previous/Next buttons and a page indicator.

We chose offset-based pagination for three reasons:

1. **Simplicity** -- it maps directly to `array[offset:offset+limit]` in Python and `OFFSET/LIMIT` in SQL. The client just tracks the current offset and increments by `limit` for the next page.
2. **Random access** -- the user can calculate any page's offset from the page number: `offset = (page - 1) * limit`. The UI shows "Page 3 of 12" and could support jumping to any page.
3. **Fits our scale** -- our dataset has thousands of categories and companies, not millions. At this scale, offset pagination performs well. The RRF fusion list is already in memory, so slicing it is $O(1)$.

The alternative -- **cursor-based pagination** -- would be more efficient for very large datasets but adds complexity. We note it as a future consideration if the dataset grows significantly.

Code walkthrough

1. Request models with offset and limit -- `search_api/models/requests.py`

```
class CategorySearchRequest(BaseModel):
    query: str = Field(..., min_length=1, description="Free-text search query")
    # limit: how many results per page (1 to 100, default 10)
    limit: int = Field(10, ge=1, le=100)
    # offset: how many results to skip (0 = first page, 10 = second page, etc.)
    offset: int = Field(0, ge=0, description="Pagination offset")
    language: str | None = Field(None)
```

```
filters: dict | None = Field(None)
rerank: bool = Field(False)
```

Pydantic validation ensures: - `limit` is between 1 and 100 (prevents absurdly large pages that could overload the server). - `offset` is non-negative (no negative skip values). - Both have sensible defaults (`limit=10`, `offset=0`) so the first page "just works" without specifying pagination params.

2. Server-side pagination in `search_table` -- `search_api/services/search.py`

```
def search_table(table, query, limit=20, offset=0, ...):
    # Over-fetch: request more results than needed for this page
    # so we have enough to paginate through
    fetch_limit = offset + limit + 10

    # ... run vector and BM25 searches with fetch_limit ...

    # RRF fusion produces the complete ranked list
    fused = fuse_rankings(vector_ids, bm25_primary_ids, bm25_secondary_ids)

    # total_count: the FULL number of fused results (all pages combined)
    total_count = len(fused)

    # Slice the fused list to get just this page's results
    page = fused[offset:offset + limit]
    #           ^^^^^ skip this many      ^^^^^ take this many

    fused_ids = [item_id for item_id, _ in page]

    # ... fetch full rows for just the IDs on this page ...

    return {
        "results": results,
        "total_count": total_count,      # client needs this for page calculation
        "offset": offset,                # echo back so client knows which page it's on
    }
```

Key design decisions:

- **Over-fetching with `fetch_limit = offset + limit + 10`**: We ask the database for more results than the current page needs. If the user is on page 3 (`offset=20`, `limit=10`), we fetch 40 results from each source. This ensures the RRF fusion has enough candidates to fill the page. The `+10` buffer accounts for potential duplicates between vector and BM25 results that might reduce the unique count after fusion.

- **Paginating after fusion, not before:** We do NOT paginate the individual vector and BM25 queries with SQL OFFSET. Instead, we fetch enough from each source, fuse them into a single ranked list, and then slice that list. This is correct because the RRF ranking is different from either source's individual ranking -- an item that was #25 in vector search and #3 in BM25 might be #5 in the fused list, so we cannot predict which source-level results will end up on which page.
- **total_count comes from the fused list:** The total is the number of unique items across all sources after fusion, not the count from any single source.

3. Client-side pagination controls -- `admin-ui/src/pages/SearchTest.jsx`

```
// State tracking the current offset
const [offset, setOffset] = useState(0);

// doSearch accepts an offset parameter, default 0
async function doSearch(searchOffset = 0) {
  // ... build request with searchOffset ...
  data = await searchCategories(query, language, limit, hasFilters, searchOffset, rerank);
  setOffset(searchOffset); // remember where we are
  setResponse(data);
}

// When the user clicks "Search", always start at page 1
function handleSearch(e) {
  e.preventDefault();
  setOffset(0); // reset to first page
  doSearch(0);
}
```

The search form always resets to page 1 on a new query. Pagination buttons navigate within the current query's results:

```
{/* Only show pagination if there are more results than one page */}
{response.total_count !== null && response.total_count > limit && (
  <div style={{ display: "flex", justifyContent: "center", gap: "0.75rem" }}>

    {/* Previous button -- goes back by one page (limit items) */}
    <button
      disabled={offset === 0 || loading}
      onClick={() => doSearch(Math.max(0, offset - limit))}
    >
      Previous
    </button>
```

```

    {/* Page indicator: "Page 3 of 12" */}
    <span>
      Page {Math.floor(offset / limit) + 1} of {Math.ceil(response.total_count / limit)}
    </span>

    {/* Next button -- advances by one page (limit items) */}
    <button
      disabled={offset + limit >= response.total_count || loading}
      onClick={() => doSearch(offset + limit)}
    >
      Next
    </button>
  </div>
)}

```

The page calculation math:

- **Current page:** `Math.floor(offset / limit) + 1` -- if `offset=20` and `limit=10`, we are on page 3.
- **Total pages:** `Math.ceil(total_count / limit)` -- if `total_count=47` and `limit=10`, there are 5 pages (`10+10+10+10+7`).
- **Previous disabled:** when `offset === 0` (already on page 1).
- **Next disabled:** when `offset + limit >= total_count` (already on the last page).

4. Result header showing position --

`admin-ui/src/pages/SearchTest.jsx`

```

{/* Shows "1-10 of 47" or "11-20 of 47" */}
<h2>
  {response.total_count != null
    ? `${offset + 1}--${offset + (response.results?.length || 0)} of ${response.total_count} Results`
    : `${response.results?.length || 0} Results`}
</h2>

```

This gives the user context about where they are in the full result set: "You are viewing results 21-30 out of 47 total."

5. Each result card shows its global rank

```

<ResultCard
  key={i}
  result={r}
  type={endpoint === "category" ? "category" : "company"}
  query={query}

```

```
rank={offset + i + 1}    // offset=20, i=0 → rank 21 (first item on page 3)
/>
```

The rank is `offset + i + 1`, not just `i + 1`, so results on page 3 show ranks 21, 22, 23... not 1, 2, 3. This prevents confusion about a result's position in the overall ranking.

Known issues with offset pagination at scale

Offset-based pagination has a well-known performance problem: **the database must compute and discard offset rows before returning limit rows**. At `offset=10,000` with `limit=10`, the database processes 10,010 rows to return 10. This gets worse with each page.

For our scale (thousands of rows, not millions), this is negligible. But for larger datasets, there are two common alternatives:

Cursor-based pagination (keyset pagination):

```
-- Instead of OFFSET, use a WHERE clause with the last seen value
SELECT * FROM results WHERE score < 0.0326 ORDER BY score DESC LIMIT 10;
```

The client sends the last result's score as the "cursor", and the database can seek directly to that position using an index. This is $O(1)$ per page regardless of depth.

Hybrid approach: Use offset for the first N pages (where it is fast) and switch to cursor-based for deeper pagination (where offset would be slow).

Our system uses offset because the typical use case is an admin user reviewing the first few pages of results, not paginating deeply through thousands of pages.

How to explain this in an interview

"We implement offset-based pagination where the client sends an offset (how many to skip) and limit (how many to return) with each search request. The server runs the full RRF fusion to get all ranked results, then slices the list at `[offset:offset+limit]` and returns that page along with a `total_count` so the UI can show 'Page 3 of 12' and enable Previous/Next buttons. We paginate after RRF fusion rather than at the SQL level because the fused ranking differs from either source's individual ranking. This approach is simple and supports random

page access, which is ideal for our dataset size. For datasets with millions of rows, we'd consider cursor-based pagination to avoid the $O(\text{offset})$ skip cost."